

**“AÑO DE LA RECUPERACIÓN Y CONSOLIDACIÓN DE LA
ECONOMÍA PERUANA”**



**Desarrollo de Aplicación Web con Impacto
Social**

Asignatura:
PROGRAMACIÓN WEB

NRC:
40076

Docente:
Taípe Miranda, Gino Joel

Ciclo:
4to

Integrantes:

- LIMACHE CUNYAS, NOEMI CIELO
- YUPANQUI SUAREZ, JESUS ANGEL

**Huancayo
2025-20**

1. Identificación de la problemática social

1.1 Descripción del problema

En la ciudad de Huancayo, muchas organizaciones sociales, comedores populares y ONGs desarrollan actividades solidarias como campañas de ayuda, donaciones o clases de refuerzo. Sin embargo, existe una gran dificultad para conectar a los voluntarios ciudadanos con estas organizaciones.

Actualmente, el proceso se realiza por redes sociales o boca a boca, lo cual genera falta de coordinación, baja participación y pérdida de oportunidades de apoyo social.

El proyecto Huancayo Solidario busca crear una plataforma web que sirva como punto de encuentro entre organizaciones sociales, voluntarios y beneficiarios, facilitando la comunicación y gestión de actividades solidarias.

1.2 Contexto local y antecedentes

Huancayo cuenta con una población diversa y numerosos sectores que enfrentan necesidades sociales, especialmente en zonas rurales o de bajos recursos. Aunque existen instituciones comprometidas, la falta de herramientas digitales limita su alcance.

Según datos de la Municipalidad Provincial de Huancayo y el INEI, más del 40% de las organizaciones sociales locales no utilizan plataformas tecnológicas para coordinar actividades. Esto genera desorganización y menor impacto en la comunidad.

1.3 Causas y consecuencias del problema

Causas principales:

- Falta de plataformas digitales para voluntariado.
- Escasa alfabetización tecnológica en sectores sociales.
- Comunicación informal entre instituciones y ciudadanos.

Consecuencias:

- Actividades sociales con baja participación.
- Desigualdad en el acceso a oportunidades de ayuda.
- Falta de transparencia en las campañas solidarias.

2. Beneficiarios y entrevistas

2.1 Identificación de beneficiarios potenciales

Los beneficiarios principales del proyecto son:

- **Organizaciones sociales:** comedores populares, parroquias, ONGs, clubes de madres.
- **Voluntarios ciudadanos:** jóvenes, estudiantes, profesionales o jubilados interesados en ayudar.
- **Personas en situación vulnerable:** adultos mayores, niños y familias que reciben apoyo.

2.2 Entrevistas a beneficiarios potenciales

Estas entrevistas permitieron confirmar la necesidad real de una herramienta digital que centralice la información y facilite la gestión solidaria.

3. Investigación complementaria

3.1 Estadísticas locales relevantes

- Según el INEI (2023), en Huancayo existen más de 150 organizaciones sociales registradas, pero solo un 35% utiliza medios digitales.
- Más del 60% de jóvenes entre 18 y 30 años estaría dispuesto a realizar voluntariado si tuviera acceso a información organizada y confiable.

3.2 Impacto social esperado

Se espera que la implementación de **Huancayo Solidario**:

- Aumente la participación de voluntarios en un 50%.
- Mejorar la transparencia de las actividades sociales.
- Fortalezca la cooperación entre instituciones locales y ciudadanos.

- Promueva una cultura de ayuda y compromiso social en la comunidad huancaína.

4. Cumplimiento ético y responsabilidad profesional

4.1 Políticas éticas y propiedad intelectual

El equipo se compromete a:

- Usar únicamente recursos y librerías libres o con licencias abiertas.
- Respetar la privacidad de los usuarios registrados.
- Citar adecuadamente las fuentes externas utilizadas.
- Garantizar que el código y diseño sean de autoría propia del grupo.

4.2 Acuerdos de trabajo y compromisos del equipo

Cada integrante asume responsabilidades claras:

- **Programación backend:** desarrollo de la API con FastAPI y base de datos.
- **Frontend:** diseño visual, animaciones y experiencia de usuario.
- **Documentación:** elaboración de informes y manuales.
- **Gestión de proyecto:** cronograma, reuniones y presentación.
Se estableció un acuerdo de comunicación semanal y registro de avances en GitHub.

5. Planificación del proyecto

5.1 Cronograma general

Semana	Actividad	Responsable
9	Identificación del problema y entrevistas	Todo el equipo
10	Elaboración del documento de propuesta	Responsable de documentación
11	Diseño de arquitectura y mockups	Diseñador web
13	Desarrollo del prototipo funcional	Programador backend y frontend
15	Entrega final del proyecto	Todo el equipo

5.2 Recursos necesarios

- Equipos de cómputo con conexión a internet.
- Cuenta GitHub para control de versiones.
- Framework FastAPI para el backend.
- HTML, CSS, JavaScript y animaciones con video de fondo para el frontend.
- Editor Visual Studio Code.

6. Rol de la ingeniería en la solución

6.1 Aporte tecnológico

La ingeniería de software aplicada en Huancayo Solidario permite transformar una problemática social en una solución tecnológica moderna.

El proyecto utiliza una arquitectura MVC (Modelo–Vista–Controlador) implementada con FastAPI en el backend para manejar los registros, autenticación de usuarios y gestión de actividades.

El frontend se desarrolla con HTML5, CSS3 y JavaScript, incluyendo efectos animados, videos de fondo y opciones interactivas que mejoran la experiencia del usuario.

El sistema permitirá a las organizaciones publicar campañas, y a los voluntarios registrarse o unirse a ellas fácilmente.

Además, se incluye un modo con video o sin video, ajustando el brillo y la música, lo cual mejora la accesibilidad para usuarios con diferentes preferencias.

Este aporte demuestra cómo la ingeniería puede unir la tecnología con la solidaridad, optimizando procesos sociales y fortaleciendo el compromiso ético del profesional con su comunidad.

6.2 Contribución social del ingeniero

El rol del ingeniero en este proyecto no solo se centra en programar, sino en entender la realidad social y crear soluciones sostenibles y éticas.

El trabajo en equipo, la responsabilidad profesional y la empatía hacia los beneficiarios son pilares fundamentales de la práctica ingenieril aplicada en este proyecto.

7. Diseño de la arquitectura de la aplicación

Se diseñó la arquitectura técnica del sistema **Huancayo Solidario** bajo el modelo **MVC (Modelo–Vista–Controlador)**, considerando tres principios fundamentales:

- **Accesibilidad:**

El diseño incorpora elementos que facilitan el uso a personas con diferentes niveles tecnológicos, como botones grandes, formularios simples, etiquetas ARIA, control de contraste y tamaño de texto, así como la opción de activar o desactivar el video y la música de fondo.

De esta manera, cualquier usuario —sin importar su experiencia tecnológica— podrá navegar y participar en campañas solidarias.

- **Escalabilidad:**

La aplicación está estructurada modularmente para poder crecer sin alterar su núcleo.

El backend, desarrollado con **FastAPI**, permite manejar múltiples rutas (endpoints) de forma independiente, mientras que la base de datos puede migrar fácilmente de **SQLite (prototipo local)** a **MySQL o PostgreSQL (producción)**.

Además, se contempla la posibilidad de usar contenedores **Docker** y despliegue en la nube (Railway, Render, etc.).

- **Sostenibilidad:**

Se incluyen **tests básicos**, un archivo **requirements.txt** con dependencias, documentación **inline (docstrings y comentarios)**, y un **README.md** que explica la instalación, ejecución y citación de librerías.

El código está versionado en **GitHub**, garantizando la trazabilidad del trabajo y las contribuciones individuales.

8. Implementación de la aplicación

Nosotros durante las semanas 11 a 14 se desarrolló el **prototipo funcional** de la aplicación utilizando herramientas de software libre y buenas prácticas de ingeniería.

8.1 Framework y tecnologías empleadas:

- **Backend:** FastAPI (Python)

Main:

```
main.py > ...
from fastapi import FastAPI, Request, Form, Depends, HTTPException
from sqlalchemy.orm import Session
from fastapi.responses import HTMLResponse, RedirectResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from fastapi.middleware.cors import CORSMiddleware
import os

print("DB USER:", os.getenv("DB_USER"))

from src.database import SessionLocal, engine
from src import models, crud, schemas

# ✅ Crear tablas
models.Base.metadata.create_all(bind=engine)

app = FastAPI(title="Huancayo Solidario")

# ✅ CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

# ✅ Static y Plantillas
app.mount("/static", StaticFiles(directory="src/static"), name="static")
templates = Jinja2Templates(directory="src/templates")

# ✅ DB Dependency
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.get("/", response_class=HTMLResponse)
def home(request: Request, db: Session = Depends(get_db)):
    return templates.TemplateResponse("index.html", {"request": request})

@app.get("/login", response_class=HTMLResponse)
def login_page(request: Request):

@app.get("/register", response_class=HTMLResponse)
def register_page(request: Request):
    return templates.TemplateResponse("loginregister.html", {"request": request, "tab": "register"})

@app.post("/register")
def register_user(
    username: str = Form(...),
    password: str = Form(...),
    db: Session = Depends(get_db)
):
    if crud.get_user_by_username(db, username):
        raise HTTPException(status_code=400, detail="Usuario ya existe")

    data = schemas.UserCreate(username=username, password=password)
    crud.create_user(db, data)

    # ✅ Redirigir al login
    return RedirectResponse("/login", status_code=303)

@app.post("/login")
def login_user(
    username: str = Form(...),
    password: str = Form(...),
    db: Session = Depends(get_db)
):
    user = crud.verify_user(db, username, password)
    if not user:
        raise HTTPException(status_code=401, detail="Credenciales incorrectas")

    return RedirectResponse("/", status_code=303)
```


Models:

```
> models.py > ...
from sqlalchemy import Column, Integer, String, Text, ForeignKey, DateTime
from sqlalchemy.orm import relationship
from datetime import datetime
from src.database import Base

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    username = Column(String, unique=True, index=True, nullable=False)
    email = Column(String, unique=True, index=True, nullable=False)
    hashed_password = Column(String, nullable=False)
    role = Column(String, default="volunteer") # 'volunteer' or 'organization'

    campaigns = relationship("Campaign", back_populates="owner")

class Campaign(Base):
    __tablename__ = "campaigns"
    id = Column(Integer, primary_key=True, index=True)
    title = Column(String, nullable=False)
    description = Column(Text)
    location = Column(String)
    date_created = Column(DateTime, default=datetime.utcnow)
    owner_id = Column(Integer, ForeignKey("users.id"))

    owner = relationship("User", back_populates="campaigns")
```

Database:

```
> database.py > ...
1  from dotenv import load_dotenv
2  load_dotenv()
3
4  from sqlalchemy import create_engine
5  from sqlalchemy.orm import sessionmaker, declarative_base
6  import os
7
8  DB_USER = os.getenv("DB_USER")
9  DB_PASS = os.getenv("DB_PASS")
10 DB_HOST = os.getenv("DB_HOST", "localhost")
11 DB_NAME = os.getenv("DB_NAME")
12
13 DB_PORT = os.getenv("DB_PORT", "3306")
14
15 SQLALCHEMY_DATABASE_URL = (
16     f"mysql+pymysql://{DB_USER}:{DB_PASS}@{DB_HOST}:{DB_PORT}/{DB_NAME}?charset=utf8mb4"
17 )
18
19
20 engine = create_engine(SQLALCHEMY_DATABASE_URL, pool_pre_ping=True)
21 SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
22 Base = declarative_base()
23
24 def get_db():
25     db = SessionLocal()
26     try:
27         yield db
28     finally:
29         db.close()
30
```

schemas:

```
schemas.py > ...
from pydantic import BaseModel, EmailStr

# ----- ACTIVITIES -----

class ActivityBase(BaseModel):
    name: str
    role: str
    activity: str

class ActivityCreate(ActivityBase):
    pass

class ActivityResponse(ActivityBase):
    id: int
    class Config:
        from_attributes = True

# ----- USERS -----

class UserBase(BaseModel):
    username: str
    email: EmailStr

class UserCreate(UserBase):
    password: str
    role: str = "volunteer" # Default role

class UserResponse(UserBase):
    id: int
    role: str

    class Config:
        from_attributes = True
```

Crud:

```
crud.py > ...
from sqlalchemy.orm import Session
from passlib.context import CryptContext
from . import models, schemas

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

# ----- USUARIOS -----

def get_user_by_email(db: Session, email: str):
    return db.query(models.User).filter(models.User.email == email).first()

def get_user_by_username(db: Session, username: str):
    return db.query(models.User).filter(models.User.username == username).first()

def create_user(db: Session, user: schemas.UserCreate):
    hashed_pass = pwd_context.hash(user.password)
    db_user = models.User(
        username=user.username,
        email=user.email,
        hashed_password=hashed_pass,
        role=user.role if hasattr(user, "role") else "volunteer"
    )
    db.add(db_user)
    db.commit()
    db.refresh(db_user)
    return db_user

def verify_password(plain, hashed):
    return pwd_context.verify(plain, hashed)

def verify_user(db: Session, username: str, password: str):
    user = get_user_by_username(db, username)
    if not user:
        return False
    if not verify_password(password, user.hashed_password):
        return False
    return user

# ----- ACTIVIDADES -----

def get_all_activities(db: Session):
    return db.query(models.Activity).all()

def get_all_activities(db: Session):
    return db.query(models.Activity).all()

def create_activity(db: Session, activity: schemas.ActivityCreate):
    db_activity = models.Activity(**activity.dict())
    db.add(db_activity)
    db.commit()
    db.refresh(db_activity)
    return db_activity

def delete_activity(db: Session, activity_id: int):
    db_activity = db.query(models.Activity).filter(models.Activity.id == activity_id).first()
    if db_activity:
        db.delete(db_activity)
        db.commit()
```

Auth:

```
auth.py > ...
from datetime import datetime, timedelta
from jose import JWTError, jwt
from typing import Optional

# Secret (en producción usar variables de entorno)
SECRET_KEY = "CAMBIA_ESTA_CLAVE_POR_1MUY_SEGURA"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 60*24 # 1 día

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None):
    to_encode = data.copy()
    expire = datetime.utcnow() + (expires_delta or timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES))
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt

def decode_access_token(token: str):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        return payload
    except JWTError:
        return None
```

- Frontend: HTML5, CSS3 y JavaScript

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Huancayo Solidario </title>
  <link rel="stylesheet" href="/static/style.css">
</head>
<body>

  <!-- VIDEO DE FONDO -->
  <video id="bg-video" autoplay muted loop playsinline>
    <source src="/static/Animated RGB 4K Wallpaper - Gravity Falls - Audio Responsive.mp4" type="video/mp4">
  </video>

  <canvas id="particles"></canvas>
  <div class="overlay"></div>

  <!-- NAVBAR FIJA -->
  <nav class="navbar">
    <h2 class="logo">♥ Huancayo Solidario</h2>
    <ul>
      <li><a href="#inicio">Inicio</a></li>
      <li><a href="#actividades">Actividades</a></li>
      <li><a href="#voluntarios">Voluntarios</a></li>
      <li><a href="/login" class="login-btn">Iniciar Sesión</a>
    </ul>
  </nav>

  <!-- HEADER PRINCIPAL -->
  <header id="inicio">
    <h1>Huancayo Solidario</h1>
    <p>Unidos por un cambio real ♥</p>
  </header>

  <!-- CONTENIDO PRINCIPAL -->
  <main>
    <!-- FORMULARIO -->
    <section id="actividades" class="form-card fade-in">
      <h2>Registra tu ayuda ♥</h2>
      <form action="/register" method="post">
        <input type="text" name="name" placeholder="Tu nombre completo" required>
        <select name="role" required>
          <option value="">Selecciona tu rol</option>
          <option value="Voluntario">Voluntario</option>
          <option value="Organización">Organización</option>
        </select>
```

```

<section id="actividades" class="form-card fade-in">
</section>

<!-- TABLA DE ACTIVIDADES -->
<section class="activities-card fade-in">
<h2>🌟 Actividades Registradas</h2>
<table>
<thead>
<tr>
<th>Nombre</th>
<th>Rol</th>
<th>Actividad</th>
<th>Acción</th>
</tr>
</thead>
<tbody>
<% for a in activities %>
<tr>
<td>{{ a.name }}</td>
<td>{{ a.role }}</td>
<td>{{ a.activity }}</td>
<td><a href="/delete/{{ a.id }}" class="delete-btn"></a></td>
</tr>
<% endfor %>
</tbody>
</table>
</section>

<!-- 📊 CONTADOR DE VOLUNTARIOS -->
<section id="voluntarios" class="counter-section fade-in">
<h2>👤 Voluntarios Activos</h2>
<div id="counter" data-target="187">0</div>
</section>
</main>

<!-- ⚙️ PIE DE PÁGINA -->
<footer>
<p>© 2025 Huancayo Solidario | Hecho con ❤️ para el bien común</p>
</footer>

<audio id="bg-music" src="/static/music.mp3" loop></audio>
<script src="/static/script.js"></script>
</body>
</html>

```

loginregister.html:

```

<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Huancayo Solidario | Iniciar Sesión</title>
<link rel="stylesheet" href="/static/loginregister.css">
</head>
<body>

<!-- 🎬 Fondo animado -->
<video id="bg-video" autoplay muted loop playsinline>
<source src="/static/Animated RGB 4K Wallpaper - Gravity Falls - Audio Responsive.mp4" type="video/mp4">
</video>
<div class="overlay"></div>

<!-- 🏠 Contenedor principal -->
<div class="container">
<h1>❤️ Huancayo Solidario</h1>
<p>Unidos por un cambio real</p>

<!-- 📄 Pestañas -->
<div class="tabs">
<button class="tab-btn active" id="login-tab">Iniciar Sesión</button>
<button class="tab-btn" id="register-tab">Registrarse</button>
</div>

<!-- 📝 Formularios -->
<div class="form-container">

<!-- Login -->
<form id="login-form" action="/login" method="post" class="form active">
<input type="text" name="username" placeholder="Nombre de usuario" required>
<input type="password" name="password" placeholder="Contraseña" required>
<button type="submit" class="btn">Entrar</button>
<p class="alt"></p>
</form>

```

Style.css:

```
src > static > style.css > ...
1  /* FONDO VIDEO Y EFECTO */
2  #bg-video {
3    position: fixed;
4    top: 0;
5    left: 0;
6    width: 100%;
7    height: 100%;
8    object-fit: cover;
9    filter: brightness(0.6) blur(2px);
10   z-index: -3;
11 }
12
13 .overlay {
14   position: fixed;
15   inset: 0;
16   background: linear-gradient(120deg, rgba(0,120,255,0.4), rgba(0,200,255,0.3));
17   z-index: -2;
18 }
19
20 #particles {
21   position: fixed;
22   top: 0;
23   left: 0;
24   width: 100%;
25   height: 100%;
26   z-index: -1;
27 }
28
29 /* NAVBAR MODERNA */
30 .navbar {
31   position: fixed;
32   top: 0;
33   left: 0;
34   width: 100%;
35   padding: 15px 40px;
36   display: flex;
37   justify-content: space-between;
38   align-items: center;
39   background: rgba(0, 0, 0, 0.35);
40   backdrop-filter: blur(12px);
41   border-bottom: 1px solid rgba(255, 255, 255, 0.15);
42   box-shadow: 0 0 25px rgba(0, 255, 255, 0.15);
43   z-index: 100;
44   animation: slideDown 0.8s ease forwards;
45 }
46
```

```

47 .navbar .logo {
48   font-size: 1.8rem;
49   font-weight: 700;
50   color: #00eaff;
51   letter-spacing: 1px;
52   text-shadow: 0 0 10px rgba(0, 255, 255, 0.6);
53   cursor: pointer;
54   transition: transform 0.3s ease;
55 }
56
57 .navbar .logo:hover {
58   transform: scale(1.05);
59 }
60
61 /* Enlaces */
62 .navbar ul {
63   list-style: none;
64   display: flex;
65   gap: 20px;
66   margin: 0;
67   padding: 0;
68 }
69
70 .navbar ul li {
71   position: relative;
72 }
73
74 .navbar ul li a {
75   text-decoration: none;
76   color: #fff;
77   font-weight: 600;
78   font-size: 1rem;
79   padding: 8px 14px;
80   transition: all 0.3s ease;
81 }
82
83 /* Efecto subrayado brillante */
84 .navbar ul li a::after {
85   content: "";
86   position: absolute;
87   left: 0;
88   bottom: -5px;
89   width: 0%;
90   height: 2px;
91   background: linear-gradient(90deg, #00c6ff, #0078ff);
92   box-shadow: 0 0 8px rgba(0, 255, 255, 0.7);
93   transition: width 0.3s ease;

```

```

95
96 .navbar ul li a: hover::after {
97   width: 100%;
98 }
99
100 .navbar ul li a: hover {
101   color: #00eaff;
102   transform: translateY(-2px);
103 }
104
105 /* 🏠 Botón "Iniciar Sesión" */
106 .navbar .login-btn {
107   background: linear-gradient(90deg, #0078ff, #00c6ff);
108   color: white !important;
109   padding: 10px 18px;
110   border-radius: 10px;
111   box-shadow: 0 0 15px rgba(0, 200, 255, 0.5);
112   transition: all 0.3s ease;
113 }
114
115 .navbar .login-btn: hover {
116   transform: scale(1.08);
117   box-shadow: 0 0 25px rgba(0, 255, 255, 0.8);
118 }
119
120 /* 📱 Responsive */
121 @media (max-width: 768px) {
122   .navbar {
123     flex-direction: column;
124     padding: 10px 20px;
125     gap: 10px;
126   }
127
128   .navbar ul {
129     flex-direction: column;
130     gap: 12px;
131     text-align: center;
132   }
133
134   .navbar ul li a {
135     font-size: 1.1rem;
136   }
137 }

```

```

139 @keyframes slideDown {
140   from {
141     opacity: 0;
142     transform: translateY(-20px);
143   }
144   to {
145     opacity: 1;
146     transform: translateY(0);
147   }
148 }
149
150 /* --- CABECERA --- */
151 header {
152   text-align: center;
153   color: white;
154   margin-top: 120px; /* 📌 más espacio por la navbar */
155   text-shadow: 0 0 10px rgba(0,0,0,0.5);
156   animation: fadeDown 1.2s ease;
157 }
158 h1 { font-size: 3rem; margin-bottom: 10px; }
159 @keyframes fadeDown {
160   from { opacity: 0; transform: translateY(-30px); }
161   to { opacity: 1; transform: translateY(0); }
162 }
163
164 /* --- CONTROLES --- */
165 .controls {
166   position: fixed;
167   top: 15px;
168   right: 15px;
169   display: flex;
170   align-items: center;
171   gap: 10px;
172   z-index: 20;
173 }
174 .ctrl-btn {
175   background: rgba(255,255,255,0.2);
176   color: white;
177   border: none;
178   padding: 8px 14px;
179   border-radius: 8px;
180   cursor: pointer;
181   font-weight: 600;
182   transition: all 0.3s;
183 }

```



```

188 main {
189   display: flex;
190   flex-direction: column;
191   align-items: center;
192   gap: 60px;
193   padding: 80px 20px;
194 }
195
196 .form-card, .activities-card {
197   background: linear-gradient(90deg, #00c6ff, #0078ff);
198   backdrop-filter: blur(10px);
199   border-radius: 20px;
200   padding: 40px;
201   width: 90%;
202   max-width: 850px;
203   color: white;
204   box-shadow: 0 0 25px #000000;
205   transition: transform 0.3s ease, box-shadow 0.3s ease;
206 }
207 .form-card:hover, .activities-card:hover {
208   transform: translateY(-8px);
209   box-shadow: 0 0 35px #000000;
210 }
211
212 /* --- FORMULARIO --- */
213 input, select, textarea {
214   width: 100%;
215   margin: 10px 0;
216   padding: 14px;
217   border-radius: 10px;
218   border: none;
219   font-size: 1rem;
220   outline: none;
221 }
222 .btn {
223   background: linear-gradient(90deg, #00c6ff, #0078ff);
224   border: none;
225   color: white;
226   padding: 14px;
227   font-size: 1.1rem;
228   font-weight: bold;
229   border-radius: 10px;
230   cursor: pointer;
231   transition: 0.3s;
232 }
233 .btn:hover {
234   background: linear-gradient(90deg, #0078ff, #00c6ff);
235   box-shadow: 0 0 20px #000000;

```

```

233 .btn:hover {
234   background: linear-gradient(90deg, #0078ff, #00c6ff);
235   box-shadow: 0 0 20px #000000;
236   transform: scale(1.03);
237 }
238
239 /* --- TABLA --- */
240 table {
241   width: 100%;
242   border-collapse: collapse;
243   margin-top: 20px;
244 }
245 th, td {
246   padding: 14px;
247   border-bottom: 1px solid #000000;
248   text-align: left;
249 }
250 th { color: white; }
251 .delete-btn {
252   color: #ff4b4b;
253   text-decoration: none;
254   font-weight: bold;
255 }
256 .delete-btn:hover { color: #ff0000; }
257
258 /* --- FOOTER --- */
259 footer {
260   text-align: center;
261   padding: 20px;
262   background: linear-gradient(90deg, #00c6ff, #0078ff);
263   backdrop-filter: blur(4px);
264   color: white;
265   margin-top: 40px;
266 }
267
268 /* --- ANIMACIONES --- */
269 .fade-in {
270   opacity: 0;
271   transform: translateY(40px);
272   animation: fadeIn 1.2s forwards;
273 }
274 @keyframes fadeIn {
275   to { opacity: 1; transform: translateY(0); }
276 }
277

```

script.js:

```
// 🎬 Control del video de fondo
const video = document.getElementById('bg-video');
const btnVideo = document.getElementById('toggle-video');
let videoActive = true;

btnVideo.addEventListener('click', () => {
  videoActive = !videoActive;
  video.style.display = videoActive ? 'block' : 'none';
  btnVideo.textContent = videoActive ? '🎬 Fondo: Activado' : '🎬 Fondo: Desactivado';
});

// 🎵 Control de música
const music = document.getElementById('bg-music');
const btnMusic = document.getElementById('toggle-music');
const volumeSlider = document.getElementById('volume');

music.volume = 0.4;
music.play();

btnMusic.addEventListener('click', () => {
  if (music.paused) {
    music.play();
    btnMusic.textContent = '🎵 Música: Activada';
  } else {
    music.pause();
    btnMusic.textContent = '🎵 Música: Desactivada';
  }
});

volumeSlider.addEventListener('input', e => {
  music.volume = e.target.value;
});

// ✨ Animación de partículas
const canvas = document.getElementById('particles');
const ctx = canvas.getContext('2d');
let particles = [];

function resize() {
  canvas.width = window.innerWidth;
  canvas.height = window.innerHeight;
}
window.addEventListener('resize', resize);
resize();
```

```
45 class Particle {
46   constructor() {
47     this.y = Math.random() * canvas.height;
48     this.size = Math.random() * 3 + 1;
49     this.speedY = Math.random() * 1 + 0.3;
50     this.color = `hsl(${Math.random()*360}, 100%, 70%)`;
51   }
52   update() {
53     this.y += this.speedY;
54     if (this.y > canvas.height) {
55       this.y = 0;
56       this.x = Math.random() * canvas.width;
57     }
58   }
59   draw() {
60     ctx.beginPath();
61     ctx.arc(this.x, this.y, this.size, 0, Math.PI*2);
62     ctx.fillStyle = this.color;
63     ctx.fill();
64   }
65 }
66
67 function initParticles() {
68   particles = [];
69   for (let i = 0; i < 100; i++) {
70     particles.push(new Particle());
71   }
72 }
73
74 function animateParticles() {
75   ctx.clearRect(0,0,canvas.width,canvas.height);
76   particles.forEach(p => {
77     p.update();
78     p.draw();
79   });
80   requestAnimationFrame(animateParticles);
81 }
82
83 initParticles();
84 animateParticles();
85
86 // 📊 Animación contador de voluntarios
87 const counter = document.getElementById("counter");
88 if (counter) {
89   const target = +counter.getAttribute("data-target");
90   let count = 0;
```

```
initParticles();
animateParticles();

// ■ Animación contador de voluntarios
const counter = document.getElementById("counter");
if (counter) {
  const target = +counter.getAttribute("data-target");
  let count = 0;

  const updateCounter = () => {
    const increment = target / 100; // velocidad de subida
    if (count < target) {
      count += increment;
      counter.textContent = Math.ceil(count);
      requestAnimationFrame(updateCounter);
    } else {
      counter.textContent = target;
    }
  };
  updateCounter();
}
```

login register.js:

```
// Cambiar entre pestañas Login / Register
const loginTab = document.getElementById('login-tab');
const registerTab = document.getElementById('register-tab');
const loginForm = document.getElementById('login-form');
const registerForm = document.getElementById('register-form');

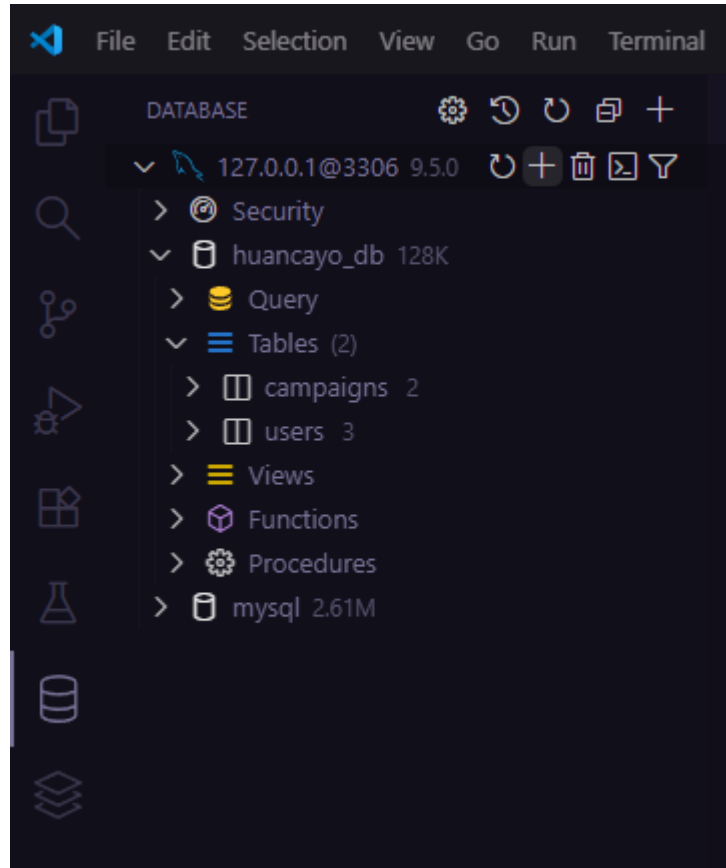
const goLogin = document.getElementById('go-login');
const goRegister = document.getElementById('go-register');

function showLogin() {
  loginForm.classList.add('active');
  registerForm.classList.remove('active');
  loginTab.classList.add('active');
  registerTab.classList.remove('active');
}

function showRegister() {
  registerForm.classList.add('active');
  loginForm.classList.remove('active');
  registerTab.classList.add('active');
  loginTab.classList.remove('active');
}

loginTab.addEventListener('click', showLogin);
registerTab.addEventListener('click', showRegister);
goLogin.addEventListener('click', showLogin);
goRegister.addEventListener('click', showRegister);
```

- **Base de Datos:** SQLite (migrable a MySQL o PostgreSQL)



```
src > .env
1  DB_USER=root
2  DB_PASS=1234
3  DB_HOST=localhost
4  DB_NAME=huancayo_db
5
```

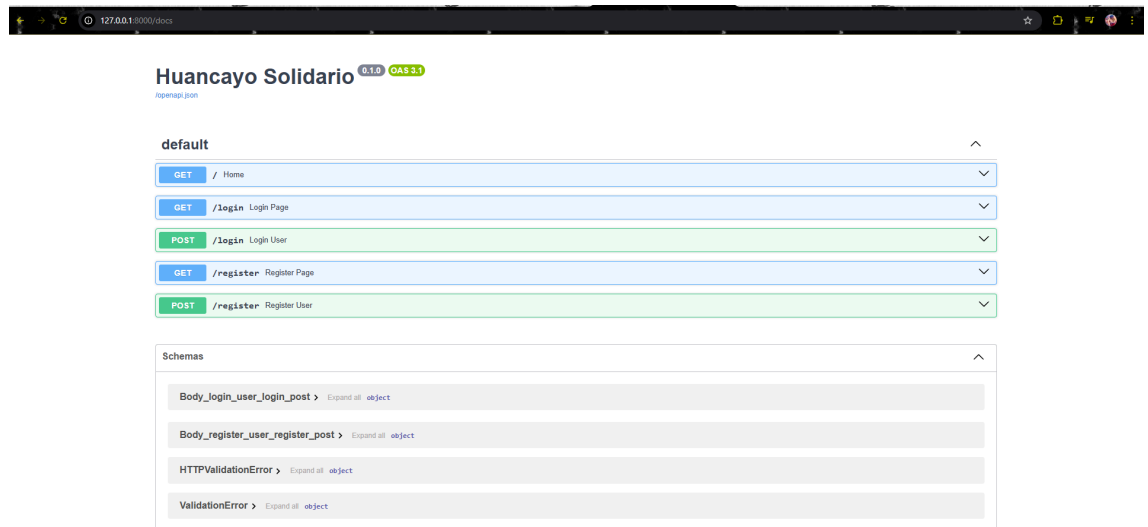


```
1  -- ☒ Crear base de datos
2  CREATE DATABASE IF NOT EXISTS huancayo_db
3      CHARACTER SET utf8mb4
4      COLLATE utf8mb4_unicode_ci;
5
6  • USE huancayo_db;
7
8  -- ☒ Borrar usuario si existe
9  DROP USER IF EXISTS 'hs_user'@'localhost';
10
11 -- ☒ Crear usuario de MySQL
12 CREATE USER 'hs_user'@'localhost' IDENTIFIED BY '1234';
13 • GRANT ALL PRIVILEGES ON huancayo_db.* TO 'hs_user'@'localhost';
14 • FLUSH PRIVILEGES;
15
16 -- ☒ Eliminar tabla si existía mal creada
17 • DROP TABLE IF EXISTS users;
18
19 -- ☒ Crear tabla correcta (sin email)
20 • CREATE TABLE users (
21     id INT AUTO_INCREMENT PRIMARY KEY,
22     username VARCHAR(100) NOT NULL UNIQUE,
23     hashed_password VARCHAR(255) NOT NULL,
24     role ENUM('volunteer','organization','admin') NOT NULL DEFAULT 'volunteer',
25
26     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
27 );
28
29 -- ☒ Insertar admin con contraseña: 1234 (bcrypt)
30 • INSERT INTO users (username, hashed_password, role)
31     VALUES (
32         'admin',
33         '$2b$12$yVQ7HLQu4AoB99jJEUbnW.y/jK1NC8SZNyIc16YKbCnX8uJeS8Jze', -- contraseña 1234
34         'admin'
35     );
```



```
1 ● USE huancayo_db;
2
3 -- Tabla users
4 ● ○ CREATE TABLE users (
5     id INT AUTO_INCREMENT PRIMARY KEY,
6     username VARCHAR(100) NOT NULL UNIQUE,
7     email VARCHAR(255) NOT NULL UNIQUE,
8     hashed_password VARCHAR(255) NOT NULL,
9     role ENUM('volunteer', 'organization', 'admin') NOT NULL DEFAULT 'volunteer',
10    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
11 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
12
13 -- Indices recomendados
14 ● CREATE INDEX idx_users_username ON users(username);
15 ● CREATE INDEX idx_users_email ON users(email);
16
17 -- Tabla campaigns
18 ● ○ CREATE TABLE campaigns (
19     id INT AUTO_INCREMENT PRIMARY KEY,
20     title VARCHAR(200) NOT NULL,
21     description TEXT,
22     location VARCHAR(200),
23     date_created TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
24     owner_id INT NOT NULL,
25     FOREIGN KEY (owner_id) REFERENCES users(id) ON DELETE CASCADE
26 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
27
28 -- Indices
29 ● CREATE INDEX idx_campaigns_owner ON campaigns(owner_id);
30 ● CREATE INDEX idx_campaigns_title ON campaigns(title);
31
1 ● USE huancayo_db;
2
3 -- Insertar usuarios de prueba (hashed_password: placeholder – en el app se guardará el hash real)
4 ● INSERT INTO users (username, email, hashed_password, role) VALUES
5     ('maría_torres', 'maria@sanvicente.org', '$2b$12$EXAMPLEHASH', 'organization'),
6     ('juan_perez', 'juan.perez@example.com', '$2b$12$EXAMPLEHASH', 'volunteer'),
7     ('admin', 'admin@huancayo.org', '$2b$12$EXAMPLEHASH', 'admin');
8
9 -- Insertar campañas de prueba
10 ● INSERT INTO campaigns (title, description, location, owner_id) VALUES
11     ('Entrega de alimentos - San Vicente', 'Reparto de víveres en barrio La Victoria', 'La Victoria - Huancayo', 1),
12     ('Clases de refuerzo gratuitas', 'Clases de matemática y comunicación para niños', 'Centro Comunitario', 1);
13
```

- **Servidor:** Uvicorn



- **Templates:** Jinja2

8.2 Buenas prácticas aplicadas:

- Uso correcto del framework FastAPI, aplicando separación por módulos (`models.py`, `schemas.py`, `crud.py`, `main.py`).
- Comentarios y docstrings para documentar funciones, clases y endpoints.
- Citación explícita de las librerías utilizadas en el archivo `README.md`.
- Registro de **commits individuales en GitHub**, donde cada miembro subió sus avances y contribuciones (backend, frontend, documentación).

9. Conclusión



La **Fase 1** permitió definir el problema, los objetivos y los requerimientos del proyecto para así poder asegurar su viabilidad técnica y conceptual, gracias a ello se establecieron las bases para el diseño y desarrollo de la aplicación. Mientras tanto en la **Fase 2** se diseña la **arquitectura del sistema**, considerando accesibilidad, escalabilidad y sostenibilidad, luego se iniciará la **implementación**, cumpliendo con el uso correcto del framework, documentación del código, citación de recursos y registro de contribuciones en Git.

10. Anexos:

Link de madifare del virtualenv:

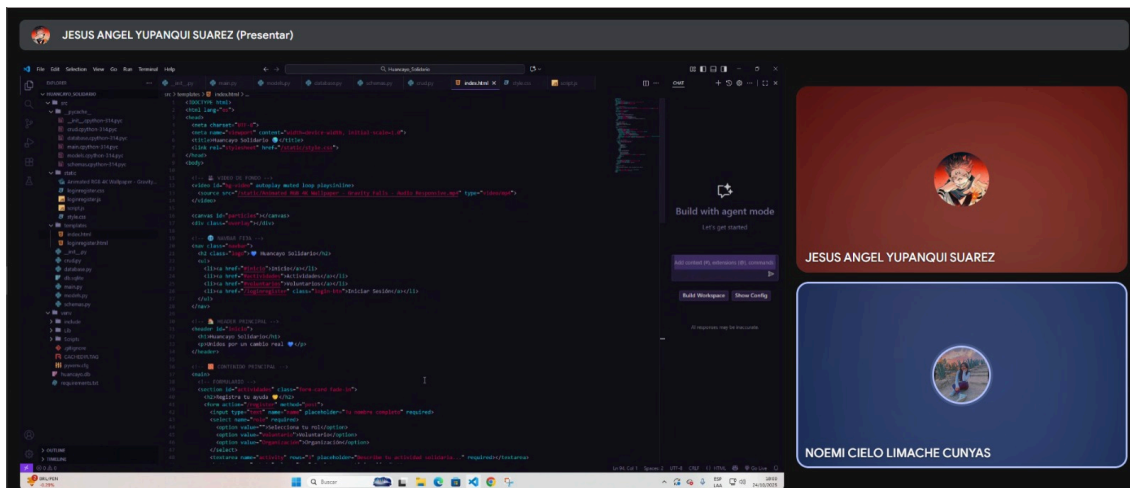
https://www.mediafire.com/file/grffr2a8ghxfmpv/Huancayo_Solidario.rar/file

Link del github:

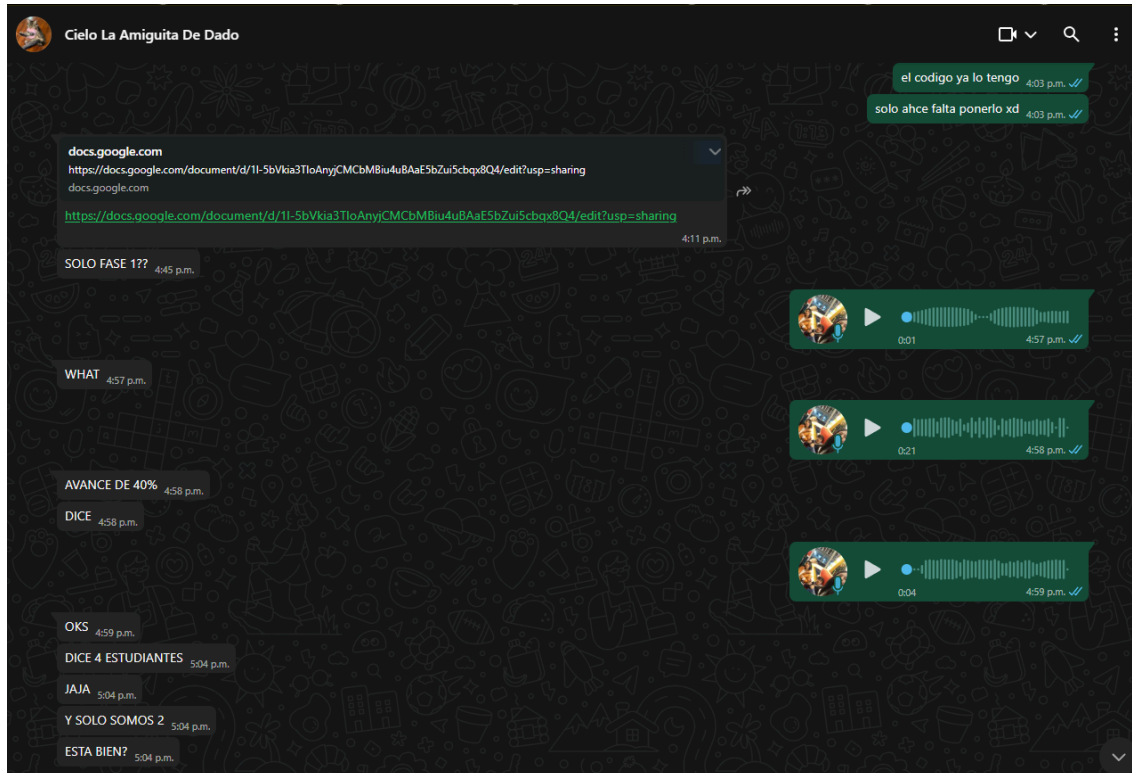
https://github.com/jesusangel13/ProgramacionWEB_HuancayoSolidario.git

11. Evidencias del trabajo:

- Meet:



- Whatsapp:



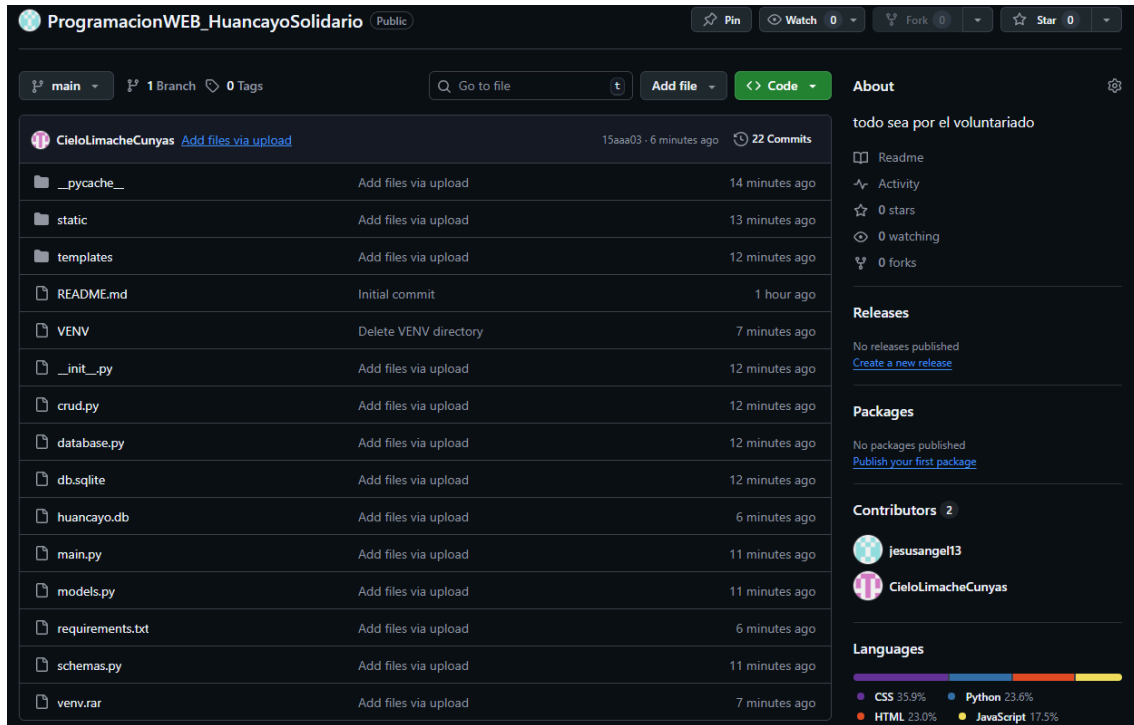
- Terminal:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\COREI7\OneDrive\Escritorio\Huancayo_Solidario> venv\Scripts\activate
(venv) PS C:\Users\COREI7\OneDrive\Escritorio\Huancayo_Solidario> uvicorn src.main:app
C:\Users\COREI7\OneDrive\Escritorio\Huancayo_Solidario\venv\Lib\site-packages\fastapi\_compat\v1.py:72: UserWarning: Cor
e Pydantic V1 functionality isn't compatible with Python 3.14 or greater.
  from pydantic.v1 import BaseConfig as BaseConfig # type: ignore[assignment]
INFO: Started server process [18528]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: 127.0.0.1:60526 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:60526 - "GET /static/music.mp3 HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:52868 - "GET /loginregister HTTP/1.1" 200 OK
INFO: 127.0.0.1:60757 - "GET /static/loginregister.js HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:52868 - "GET /static/loginregister.css HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:55341 - "GET /static/Animated%20RGB%204K%20Wallpaper%20-%20Gravity%20Falls%20-%20Audio%20Responsive.
mp4 HTTP/1.1" 206 Partial Content
INFO: 127.0.0.1:63780 - "GET /static/Animated%20RGB%204K%20Wallpaper%20-%20Gravity%20Falls%20-%20Audio%20Responsive.
mp4 HTTP/1.1" 206 Partial Content
```

- **Git Hub:**



The screenshot shows a GitHub repository page for 'ProgramacionWEB_HuancayoSolidario'. The repository is public and has 22 commits. The commit history table lists the following files and their commit times:

File	Commit Message	Time Ago
__pycache__	Add files via upload	14 minutes ago
static	Add files via upload	13 minutes ago
templates	Add files via upload	12 minutes ago
README.md	Initial commit	1 hour ago
VENV	Delete VENV directory	7 minutes ago
__init__.py	Add files via upload	12 minutes ago
crud.py	Add files via upload	12 minutes ago
database.py	Add files via upload	12 minutes ago
db.sqlite	Add files via upload	12 minutes ago
huancayo.db	Add files via upload	6 minutes ago
main.py	Add files via upload	11 minutes ago
models.py	Add files via upload	11 minutes ago
requirements.txt	Add files via upload	6 minutes ago
schemas.py	Add files via upload	11 minutes ago
venv.rar	Add files via upload	7 minutes ago

The right sidebar contains the following sections:

- About:** todo sea por el voluntariado. Includes links for Readme, Activity, 0 stars, 0 watching, and 0 forks.
- Releases:** No releases published. Link: [Create a new release](#).
- Packages:** No packages published. Link: [Publish your first package](#).
- Contributors:** 2 contributors: [jesusangel13](#) and [CieloLimacheCunyas](#).
- Languages:** A bar chart showing the following language distribution:
 - CSS: 35.9%
 - Python: 23.6%
 - HTML: 23.0%
 - JavaScript: 17.5%

Diagrama:

